DASO：从压缩方法，到部署决策

Deployment-Aware Strategy Optimization · 面向部署的压缩策略优化

可选方法越来越多

- BF16
- Dense NVFP4
- Sparse BF16
- Sparse NVFP4
- W4A16 Marlin

实际部署，需要回答三个问题

01 **在哪些模块用?**

不同层与模块，对压缩的敏感度不同。

02 **Prefill 与 Decode 如何搭配?**

同一种方法，在不同推理阶段的收益不同。

03 **质量预算下，能快多少?**

需要面向具体模型、硬件和负载做决策。

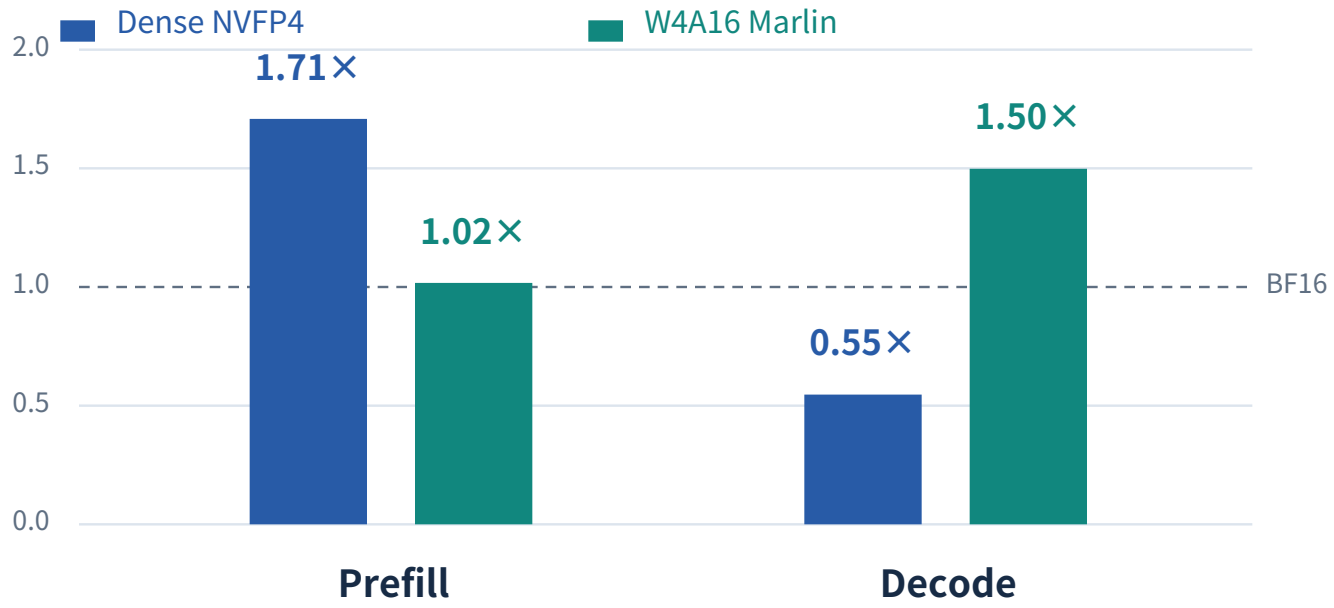
> **研究目标：在质量损失预算内，选择更快的细粒度压缩策略。**

依据：最终论文 Introduction / Framework Overview。

一个方法，难以覆盖所有推理阶段

RTX 5090 · Llama-2-7B-Chat：均匀 NVFP4 与 W4A16 呈现互补收益

相对 BF16 的阶段加速（×）



阶段差异

需要分别选择
Prefill 与 Decode 的方法。

模块差异

保留模块之间的敏感度差异，
避免对所有模块统一压缩。

启发：让各模块、各推理阶段分别选择合适的方法。

数据：最终主表；Prefill 为 TTFT 加速，Decode 为 TPOT 加速；BF16 = 1.0×。

建立可比较的时延与质量代价

面向每个“模块 × 推理阶段 × 候选方法”，估计速度收益与质量损失

时延建模：硬件与形状感知

方法感知的 Roofline 先验

按方法区分计算、访存和固定开销，
建立可跨形状比较的时延基线。

用实测数据修正预测偏差

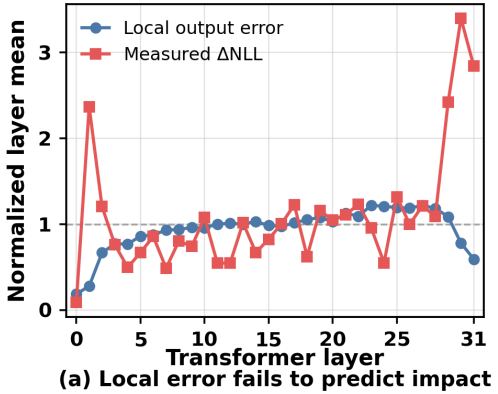
结合全局趋势与邻近形状，
修正真实内核效率造成的偏差。

输出：模块 × 阶段 × 方法的时延估计

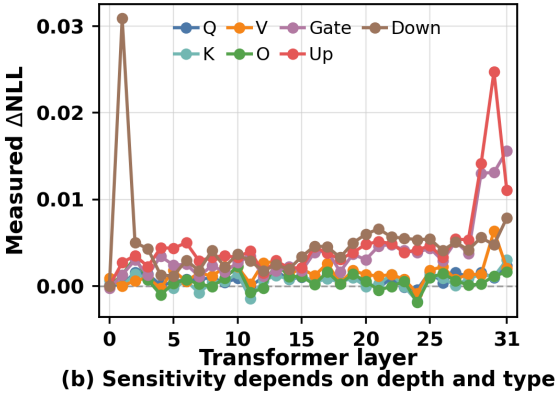
质量建模：从局部扰动到 ΔNLL

局部输出扰动 + 少量策略级 NLL

按方法、层深、模块类型与阶段校准，
将局部误差映射为整体质量损失。



(a) Local error fails to predict impact



(b) Sensitivity depends on depth and type

输出统一的时延—质量代价，支持跨方法的策略比较。

依据：最终论文 Latency / Quality Modeling；右下为论文质量建模原图。

预算约束优化与实际推理执行

将预测代价转化为模块级、阶段级策略，并落实到 vLLM 运行时

策略优化：在质量预算内最小化时延

每个模块选择一对 Prefill / Decode 方法

1 按执行次数汇总阶段时延

2 以 ΔNLL 限制总质量损失

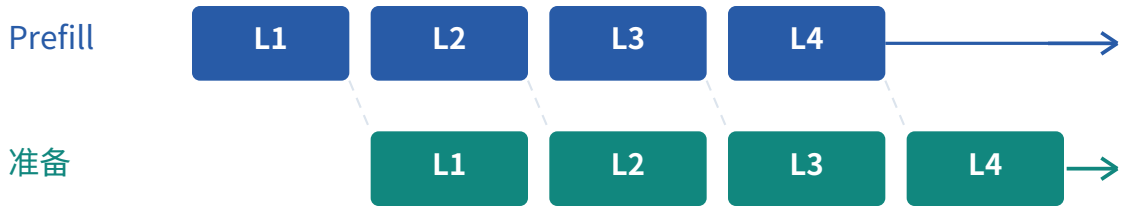
3 动态规划选择各模块的方法对

扫描质量预算 → 生成不同折中策略

运行时：让异构策略真正跑起来

按模块与推理阶段分派不同计算后端。

逐层计算与 Decode 权重准备重叠



时序示意；方块不表示实际持续时间。

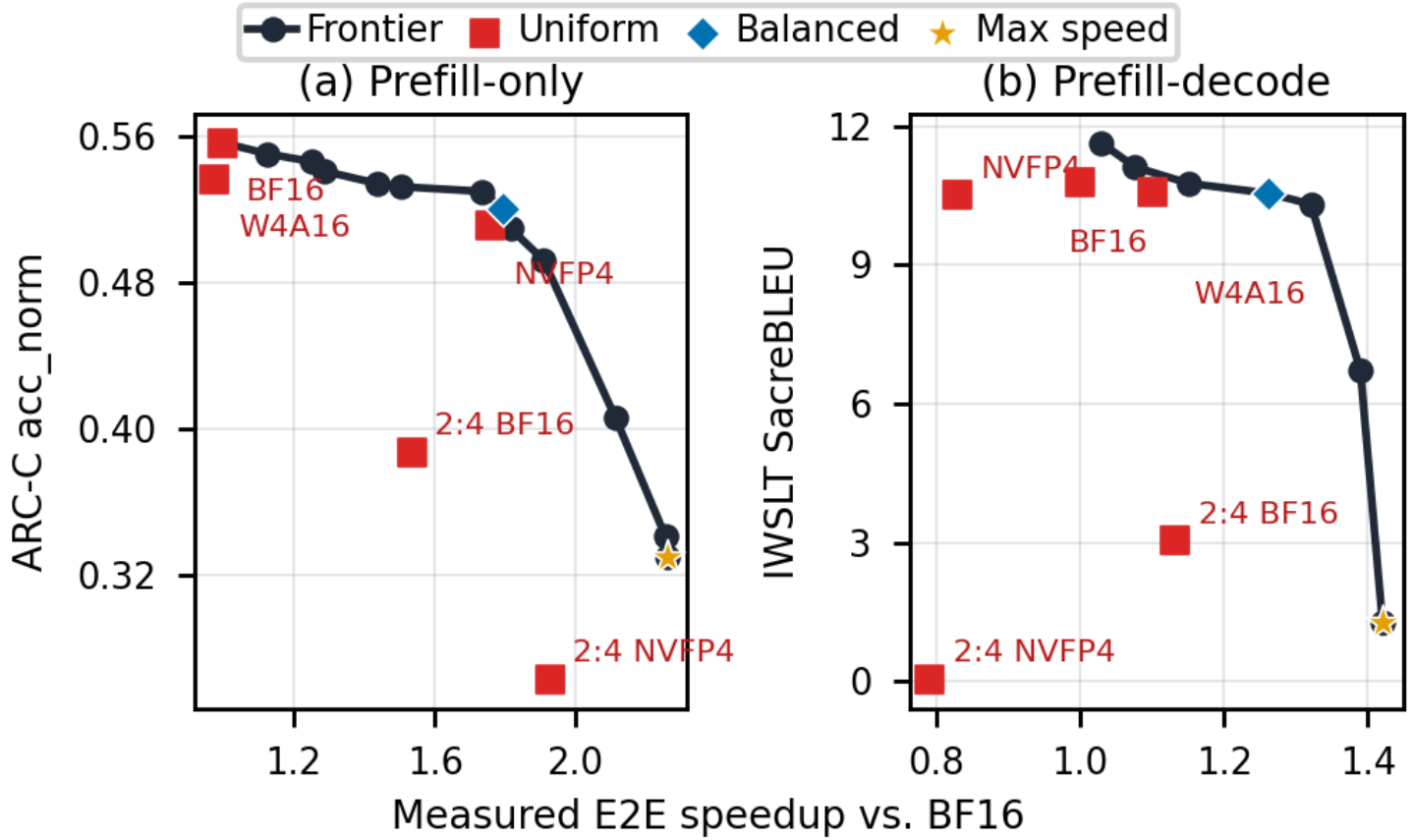
通过异步预取，减少阶段切换的全局等待。

质量预算决定策略，运行时将策略转化为实际推理加速。

依据：最终论文 Strategy Optimization and Deployment；右侧时序为解释性示意。

实验结果：速度收益与质量折中

原图：Llama-3.1-8B-Instruct · RTX 5090；右侧摘要来自最终主表



论文中的主要结果

2.26×

最大 Prefill 加速 · Llama-3.1

1.55×

最大 E2E 加速 · Llama-2

1.14–1.26×

Balanced 的 E2E 加速

最大加速使用宽松质量预算；
Balanced 体现较保守的质量折中。

左：论文质量—速度原图；右：最终主表（两模型，RTX 5090）。B=8，S=2048，生成长度 T=64。